

tinykernel

構築: Doxygen 1.18.0



|                                                                                                       |           |
|-------------------------------------------------------------------------------------------------------|-----------|
| <b>1 ディレクトリ索引</b>                                                                                     | <b>1</b>  |
| 1.1 ディレクトリ                                                                                            | 1         |
| <b>2 名前空間索引</b>                                                                                       | <b>3</b>  |
| 2.1 名前空間一覧                                                                                            | 3         |
| <b>3 クラス索引</b>                                                                                        | <b>5</b>  |
| 3.1 クラス一覧                                                                                             | 5         |
| <b>4 ファイル索引</b>                                                                                       | <b>7</b>  |
| 4.1 ファイル一覧                                                                                            | 7         |
| <b>5 ディレクトリ詳解</b>                                                                                     | <b>9</b>  |
| 5.1 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include ディレクトリ<br>リファレンス            | 9         |
| 5.2 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/port/m33 ディレク<br>トリリファレンス       | 9         |
| 5.3 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/port ディレクトリ<br>リファレンス           | 9         |
| 5.4 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src ディレクトリリフ<br>ァレンス                | 9         |
| 5.5 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel ディレクトリリファレ<br>ンス                    | 10        |
| 5.6 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel ディ<br>レクトリリファレンス | 10        |
| <b>6 名前空間詳解</b>                                                                                       | <b>11</b> |
| 6.1 tk 名前空間                                                                                           | 11        |
| 6.1.1 型定義詳解                                                                                           | 12        |
| 6.1.1.1 BinarySemaphore                                                                               | 12        |
| 6.1.1.2 task_mask_t                                                                                   | 12        |
| 6.1.1.3 TaskFn                                                                                        | 13        |
| 6.1.2 列挙型詳解                                                                                           | 13        |
| 6.1.2.1 Prio                                                                                          | 13        |
| 6.1.3 関数詳解                                                                                            | 13        |
| 6.1.3.1 create()                                                                                      | 13        |
| 6.1.3.2 exit()                                                                                        | 13        |
| 6.1.3.3 lock()                                                                                        | 14        |
| 6.1.3.4 now_ms()                                                                                      | 14        |
| 6.1.3.5 preempt_disable()                                                                             | 14        |
| 6.1.3.6 preempt_enable()                                                                              | 15        |
| 6.1.3.7 publish() [1/2]                                                                               | 15        |
| 6.1.3.8 publish() [2/2]                                                                               | 15        |
| 6.1.3.9 sleep_ms()                                                                                    | 16        |
| 6.1.3.10 start()                                                                                      | 16        |
| 6.1.3.11 unlock()                                                                                     | 16        |

|                                                       |    |
|-------------------------------------------------------|----|
| 6.1.3.12 yield()                                      | 17 |
| 6.2 tk::detail 名前空間                                   | 17 |
| 6.2.1 関数詳解                                            | 17 |
| 6.2.1.1 block_current_task_locked()                   | 17 |
| 6.2.1.2 current_task_slot()                           | 17 |
| 6.2.1.3 find_highest_prio_waiter()                    | 17 |
| 6.2.1.4 sched_lock()                                  | 18 |
| 6.2.1.5 sched_unlock()                                | 18 |
| 6.2.1.6 static_register()                             | 18 |
| 6.2.1.7 wake_task_locked()                            | 18 |
| <b>7 クラス詳解</b>                                        | 19 |
| 7.1 tk::Actor< MsgType, QueueSize > クラステンプレート         | 19 |
| 7.1.1 詳解                                              | 19 |
| 7.1.2 構築子と解体子                                         | 20 |
| 7.1.2.1 Actor()                                       | 20 |
| 7.1.2.2 ~Actor()                                      | 20 |
| 7.1.3 関数詳解                                            | 20 |
| 7.1.3.1 handle()                                      | 20 |
| 7.1.3.2 on_message()                                  | 20 |
| 7.1.3.3 on_start()                                    | 21 |
| 7.1.3.4 post() [1/3]                                  | 21 |
| 7.1.3.5 post() [2/3]                                  | 21 |
| 7.1.3.6 post() [3/3]                                  | 22 |
| 7.1.3.7 start()                                       | 22 |
| 7.2 tk::EventHub< MsgType, MaxSubscribers > クラステンプレート | 22 |
| 7.2.1 詳解                                              | 23 |
| 7.2.2 関数詳解                                            | 23 |
| 7.2.2.1 publish()                                     | 23 |
| 7.2.2.2 subscribe()                                   | 23 |
| 7.2.2.3 subscriber_count()                            | 24 |
| 7.2.2.4 unsubscribe()                                 | 24 |
| 7.3 FpuCtx 構造体                                        | 24 |
| 7.3.1 メンバ詳解                                           | 25 |
| 7.3.1.1 s                                             | 25 |
| 7.4 tk::LockGuard< Lockable > クラステンプレート               | 25 |
| 7.4.1 詳解                                              | 25 |
| 7.4.2 構築子と解体子                                         | 25 |
| 7.4.2.1 LockGuard() [1/2]                             | 25 |
| 7.4.2.2 ~LockGuard()                                  | 26 |
| 7.4.2.3 LockGuard() [2/2]                             | 26 |
| 7.4.3 関数詳解                                            | 26 |
| 7.4.3.1 operator=()                                   | 26 |

|                                                                                                                     |           |
|---------------------------------------------------------------------------------------------------------------------|-----------|
| 7.5 tk::Message 構造体 . . . . .                                                                                       | 26        |
| 7.5.1 詳解 . . . . .                                                                                                  | 27        |
| 7.5.2 メンバ詳解 . . . . .                                                                                               | 27        |
| 7.5.2.1 arg . . . . .                                                                                               | 27        |
| 7.5.2.2 id . . . . .                                                                                                | 27        |
| 7.6 tk::Mutex クラス . . . . .                                                                                         | 27        |
| 7.6.1 詳解 . . . . .                                                                                                  | 28        |
| 7.6.2 構築子と解体子 . . . . .                                                                                             | 28        |
| 7.6.2.1 Mutex() . . . . .                                                                                           | 28        |
| 7.6.3 関数詳解 . . . . .                                                                                                | 28        |
| 7.6.3.1 is_locked() . . . . .                                                                                       | 28        |
| 7.6.3.2 lock() . . . . .                                                                                            | 28        |
| 7.6.3.3 try_lock() . . . . .                                                                                        | 28        |
| 7.6.3.4 unlock() . . . . .                                                                                          | 29        |
| 7.7 tk::PreemptGuard クラス . . . . .                                                                                  | 29        |
| 7.7.1 詳解 . . . . .                                                                                                  | 29        |
| 7.7.2 構築子と解体子 . . . . .                                                                                             | 29        |
| 7.7.2.1 PreemptGuard() [1/2] . . . . .                                                                              | 29        |
| 7.7.2.2 ~PreemptGuard() . . . . .                                                                                   | 29        |
| 7.7.2.3 PreemptGuard() [2/2] . . . . .                                                                              | 30        |
| 7.7.3 関数詳解 . . . . .                                                                                                | 30        |
| 7.7.3.1 operator=() . . . . .                                                                                       | 30        |
| 7.8 tk::Semaphore クラス . . . . .                                                                                     | 30        |
| 7.8.1 詳解 . . . . .                                                                                                  | 30        |
| 7.8.2 構築子と解体子 . . . . .                                                                                             | 30        |
| 7.8.2.1 Semaphore() . . . . .                                                                                       | 30        |
| 7.8.3 関数詳解 . . . . .                                                                                                | 31        |
| 7.8.3.1 acquire() . . . . .                                                                                         | 31        |
| 7.8.3.2 count() . . . . .                                                                                           | 31        |
| 7.8.3.3 release() . . . . .                                                                                         | 31        |
| 7.8.3.4 try_acquire() . . . . .                                                                                     | 31        |
| 7.9 tk::TaskHandle 構造体 . . . . .                                                                                    | 32        |
| 7.9.1 詳解 . . . . .                                                                                                  | 32        |
| 7.9.2 関数詳解 . . . . .                                                                                                | 32        |
| 7.9.2.1 is_valid() . . . . .                                                                                        | 32        |
| 7.9.3 メンバ詳解 . . . . .                                                                                               | 32        |
| 7.9.3.1 slot . . . . .                                                                                              | 32        |
| <b>8 ファイル詳解 . . . . .</b>                                                                                           | <b>33</b> |
| 8.1 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/↵<br>actor.hpp ファイル . . . . .  | 33        |
| 8.2 actor.hpp . . . . .                                                                                             | 34        |
| 8.3 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/↵<br>kernel.hpp ファイル . . . . . | 36        |

|                                                                                                                  |    |
|------------------------------------------------------------------------------------------------------------------|----|
| 8.3.1 マクロ定義詳解 . . . . .                                                                                          | 37 |
| 8.3.1.1 TK_DEFAULT_STACK_SIZE . . . . .                                                                          | 37 |
| 8.3.1.2 TK_MAX_TASKS . . . . .                                                                                   | 38 |
| 8.3.1.3 TK_TASK . . . . .                                                                                        | 38 |
| 8.3.1.4 TK_TICKS_PER_SEC . . . . .                                                                               | 38 |
| 8.3.1.5 TK_TIME_SLICE_MS . . . . .                                                                               | 38 |
| 8.4 kernel.hpp . . . . .                                                                                         | 39 |
| 8.5 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/<br>sync.hpp ファイル . . . . . | 40 |
| 8.6 sync.hpp . . . . .                                                                                           | 40 |
| 8.7 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/port/m33/port_<br>m33.c ファイル . . . . .     | 42 |
| 8.7.1 マクロ定義詳解 . . . . .                                                                                          | 43 |
| 8.7.1.1 FPCCR_ASPEN . . . . .                                                                                    | 43 |
| 8.7.1.2 FPCCR_LSPEN . . . . .                                                                                    | 43 |
| 8.7.2 関数詳解 . . . . .                                                                                             | 43 |
| 8.7.2.1 tk_port_init() . . . . .                                                                                 | 43 |
| 8.8 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/port/m33/port_<br>m33.h ファイル . . . . .     | 43 |
| 8.8.1 関数詳解 . . . . .                                                                                             | 43 |
| 8.8.1.1 tk_port_init() . . . . .                                                                                 | 43 |
| 8.8.1.2 tk_port_make_frame() . . . . .                                                                           | 43 |
| 8.8.1.3 tk_port_start() . . . . .                                                                                | 44 |
| 8.9 port_m33.h . . . . .                                                                                         | 44 |
| 8.10 C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/scheduler.cpp フ<br>ァイル . . . . .          | 44 |
| 8.10.1 関数詳解 . . . . .                                                                                            | 45 |
| 8.10.1.1 isr_systick() . . . . .                                                                                 | 45 |
| 8.10.1.2 tk_switch_context() . . . . .                                                                           | 45 |
| 8.10.1.3 tk_task_return_trampoline() . . . . .                                                                   | 45 |
| 8.10.2 変数詳解 . . . . .                                                                                            | 45 |
| 8.10.2.1 tk_fpu_scratch . . . . .                                                                                | 45 |
| Index . . . . .                                                                                                  | 47 |

## Chapter 1

# ディレクトリ索引

### 1.1 ディレクトリ

|               |    |
|---------------|----|
| include       | 9  |
| tinykernel    | 10 |
| actor.hpp     | 33 |
| kernel.hpp    | 36 |
| sync.hpp      | 40 |
| m33           | 9  |
| port_m33.c    | 42 |
| port_m33.h    | 43 |
| port          | 9  |
| m33           | 9  |
| port_m33.c    | 42 |
| port_m33.h    | 43 |
| src           | 9  |
| port          | 9  |
| m33           | 9  |
| port_m33.c    | 42 |
| port_m33.h    | 43 |
| scheduler.cpp | 44 |
| tinykernel    | 10 |
| include       | 9  |
| tinykernel    | 10 |
| actor.hpp     | 33 |
| kernel.hpp    | 36 |
| sync.hpp      | 40 |
| src           | 9  |
| port          | 9  |
| m33           | 9  |
| port_m33.c    | 42 |
| port_m33.h    | 43 |
| scheduler.cpp | 44 |
| tinykernel    | 10 |
| actor.hpp     | 33 |
| kernel.hpp    | 36 |
| sync.hpp      | 40 |



## Chapter 2

# 名前空間索引

### 2.1 名前空間一覧

全名前空間の一覧です。

|                      |    |
|----------------------|----|
| tk . . . . .         | 11 |
| tk::detail . . . . . | 17 |



## Chapter 3

# クラス索引

### 3.1 クラス一覧

クラス・構造体・共用体・インターフェースの一覧です。

|                                                                    |    |
|--------------------------------------------------------------------|----|
| tk::Actor< MsgType, QueueSize >                                    |    |
| メッセージ駆動型タスク (アクター) の基底クラス . . . . .                                | 19 |
| tk::EventHub< MsgType, MaxSubscribers >                            |    |
| メッセージ型ごとの超軽量 Pub/Sub ディスパッチャ . . . . .                             | 22 |
| FpuCtx . . . . .                                                   | 24 |
| tk::LockGuard< Lockable >                                          |    |
| ミューテックスのロックをスコープ単位で安全に管理する RAII クラス (std::lock_guard 相当) . . . . . | 25 |
| tk::Message                                                        |    |
| デフォルトの軽量メッセージ構造体 . . . . .                                         | 26 |
| tk::Mutex                                                          |    |
| 所有権・優先度順ウェイクアップ・RAII 対応のミューテックス (排他ロック) . . . . .                  | 27 |
| tk::PreemptGuard                                                   |    |
| プリエンプション禁止をスコープ単位で安全に管理する RAII クラス . . . . .                       | 29 |
| tk::Semaphore                                                      |    |
| 計数セマフォ (Counting Semaphore (p.30)) / バイナリセマフォ . . . . .            | 30 |
| tk::TaskHandle                                                     |    |
| タスクの識別ハンドル . . . . .                                               | 32 |



## Chapter 4

# ファイル索引

### 4.1 ファイル一覧

ファイル一覧です。

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/actor.hpp  | 33 |
| C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/kernel.hpp | 36 |
| C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/sync.hpp   | 40 |
| C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/scheduler.cpp             | 44 |
| C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/port/m33/port_m33.c       | 42 |
| C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/src/port/m33/port_m33.h       | 43 |



## Chapter 5

# ディレクトリ詳解

5.1 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵  
tinykernel/include **ディレクトリリファレンス**

### ディレクトリ

- ディレクトリ tinykernel

5.2 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵  
tinykernel/src/port/m33 **ディレクトリリファレンス**

### ファイル

- ファイル port\_m33.c
- ファイル port\_m33.h

5.3 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵  
tinykernel/src/port **ディレクトリリファレンス**

### ディレクトリ

- ディレクトリ m33

5.4 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵  
tinykernel/src **ディレクトリリファレンス**

### ディレクトリ

- ディレクトリ port

## ファイル

- ファイル scheduler.cpp

## 5.5 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel ディレクトリリファレンス

### ディレクトリ

- ディレクトリ include
- ディレクトリ src

## 5.6 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/ tinykernel/include/tinykernel ディレクトリリファレンス

## ファイル

- ファイル actor.hpp
- ファイル kernel.hpp
- ファイル sync.hpp

## Chapter 6

# 名前空間詳解

### 6.1 tk 名前空間

#### 名前空間

- namespace detail

#### クラス

- class Actor  
メッセージ駆動型タスク (アクター) の基底クラス
- class EventHub  
メッセージ型ごとの超軽量 Pub/Sub ディスパッチャ
- class LockGuard  
ミューテックスのロックをスコープ単位で安全に管理する RAII クラス (std::lock\_guard 相当)
- struct Message  
デフォルトの軽量メッセージ構造体
- class Mutex  
所有権・優先度順ウェイクアップ・RAII 対応のミューテックス (排他ロック)
- class PreemptGuard  
プリエンプション禁止をスコープ単位で安全に管理する RAII クラス
- class Semaphore  
計数セマフォ (Counting Semaphore (p.30)) / バイナリセマフォ
- struct TaskHandle  
タスクの識別ハンドル

#### 型定義

- using TaskFn = void (\*)(void\* arg)
- using task\_mask\_t = uint32\_t
- using BinarySemaphore = Semaphore  
バイナリセマフォ (初期値 1, 最大 1) の型エイリアス

## 列挙型

- `enum Prio : uint32_t { PRIO_REALTIME = 1, PRIO_HIGH = 5, PRIO_NORMAL = 9, PRIO_LOW = 13 }`  
タスクの優先度レベル

## 関数

- `template<typename MsgType> size_t publish (const MsgType &msg)`  
メッセージの型を指定して、購読しているすべての Actor (p. 19) へ同報配信します
- `size_t publish (uint32_t id, uintptr_t arg=0)`  
`tk::Message` (p. 26) 型の同報配信ショートカット
- `TaskHandle create ( TaskFn fn, void *arg=nullptr, uint32_t prio= PRIO_NORMAL)`  
タスクを生成・登録します
- `void start ()`  
`tinykernel` スケジューラを起動します
- `void yield ()`  
現在実行中のタスクを一時中断し、同優先度の他のタスクへ CPU を譲渡します
- `void sleep_ms (uint32_t ms)`  
現在のタスクを指定時間スリープさせます
- `void exit (int code=0)`  
現在のタスクを終了・破棄します
- `uint64_t now_ms ()`  
システム起動時からの経過時間をミリ秒単位で取得します
- `bool lock ()`  
ハードウェアスピンロックと全割り込み禁止によるクリティカルセクションに入ります
- `void unlock (bool was_enabled)`  
`tk::lock()` (p. 14) で入ったクリティカルセクションを抜けます
- `void preempt_disable ()`  
自コアのタスクスイッチ (プリエンプション) を一時的に禁止します
- `void preempt_enable ()`  
自コアのタスクスイッチ (プリエンプション) を再開します

### 6.1.1 型定義詳解

#### 6.1.1.1 BinarySemaphore

```
using tk::BinarySemaphore = Semaphore
```

バイナリセマフォ (初期値 1, 最大 1) の型エイリアス

#### 6.1.1.2 task\_mask\_t

```
using tk::task_mask_t = uint32_t
```

## 6.1.1.3 TaskFn

```
using tk::TaskFn = void (*)(void* arg)
```

タスク関数のシグネチャ (void func(void\* arg))

## 6.1.2 列挙型詳解

## 6.1.2.1 Prio

```
enum tk::Prio : uint32_t
```

タスクの優先度レベル

数値が小さいほど高優先度です。0..15 の範囲で指定します (15 はアイドルタスク専用)。

## 列挙値

|               |               |
|---------------|---------------|
| PRIO_REALTIME | リアルタイム最優先     |
| PRIO_HIGH     | 高優先度          |
| PRIO_NORMAL   | 通常優先度 (デフォルト) |
| PRIO_LOW      | 低優先度          |

## 6.1.3 関数詳解

## 6.1.3.1 create()

```
tk::TaskHandle tk::create (
    TaskFn fn,
    void * arg = nullptr,
    uint32_t prio = PRIO_NORMAL)
```

タスクを生成・登録します

tk::start() (p. 16) を呼び出す前に登録する必要があります。

## 引数

|      |                                        |
|------|----------------------------------------|
| fn   | タスク関数ポインタ (void (*)(void* arg))        |
| arg  | タスク関数に渡すユーザー引数ポインタ (省略時: nullptr)      |
| prio | タスクの優先度 (省略時: tk::PRIO_NORMAL (p. 13)) |

戻り値

TaskHandle (p. 32) 生成されたタスクのハンドル。登録失敗時は is\_valid() が false

構築: Doxygen

## 6.1.3.2 exit()

## 引数

|      |                    |
|------|--------------------|
| code | 終了コード (予約用、省略時: 0) |
|------|--------------------|

## 覚え書き

タスク関数の末尾に到達した場合は自動的に呼ばれます。

### 6.1.3.3 lock()

bool tk::lock ()

ハードウェアスピンのロックと全割り込み禁止によるクリティカルセクションに入ります

## 戻り値

true 呼び出し前の割り込みが有効だった場合  
false 呼び出し前の割り込みが無効だった場合

## 警告

長時間のロックは避けてください。数マイクロ秒以内のレジスタ操作等に使用します。

### 6.1.3.4 now\_ms()

uint64\_t tk::now\_ms ()

システム起動時からの経過時間をミリ秒単位で取得します

## 戻り値

uint64\_t 経過ミリ秒数 (分解能: 1 / TK\_TICKS\_PER\_SEC 秒)

### 6.1.3.5 preempt\_disable()

void tk::preempt\_disable ()

自コアのタスクスイッチ (プリエンプション) を一時的に禁止します

割り込み (SysTick やタイマー、NVIC) は停止しないため、通信クロックや時刻更新を阻害することなく、自タスクの処理をアトミックに実行できます。ネスト (入れ子) 呼び出しに対応しています。

## 参照

tk::PreemptGuard (p. 29), tk::preempt\_enable() (p. 15)

## 6.1.3.6 preempt\_enable()

```
void tk::preempt_enable ()
```

自コアのタスクスイッチ (プリエンプション) を再開します

ネストカウンタが 0 に戻った時点で、保留されていたタスクスイッチが即座に実行されます。

参照

tk::PreemptGuard (p. 29), tk::preempt\_disable() (p. 14)

## 6.1.3.7 publish() [1/2]

```
template<typename MsgType>
size_t tk::publish (
    const MsgType & msg) [inline]
```

メッセージの型を指定して、購読しているすべての Actor (p. 19) へ同報配信します

送信側は Actor (p. 19) のインスタンスや名前を知る必要がありません (完全疎結合)。メッセージの実体は各 Actor (p. 19) のキューへ値コピーされるため、スタック上のローカル変数でも安全に渡せます。

## テンプレート引数

|         |            |
|---------|------------|
| MsgType | 配信するメッセージ型 |
|---------|------------|

## 引数

|     |           |
|-----|-----------|
| msg | 送信するメッセージ |
|-----|-----------|

戻り値

size\_t 配信に成功した Actor (p. 19) の数

```
MyEvent ev{1, "Button Pressed"};
tk::publish(ev); // MyEvent を受け付けるすべての Actor に届く
```

## 6.1.3.8 publish() [2/2]

```
size_t tk::publish (
    uint32_t id,
    uintptr_t arg = 0) [inline]
```

tk::Message (p. 26) 型の同報配信ショートカット

**引数**

|     |             |
|-----|-------------|
| id  | メッセージID     |
| arg | 引数 (省略時: 0) |

戻り値

size\_t 配信に成功した Actor (p.19) の数

**6.1.3.9 sleep\_ms()**

```
void tk::sleep_ms (  
    uint32_t ms)
```

現在のタスクを指定時間スリープさせます

スリープ中はタスクが休止状態になり、CPU は他のタスクへ自動的に割り当てられます (CPU 消費ゼロ)。

**引数**

|    |                |
|----|----------------|
| ms | スリープする時間 (ミリ秒) |
|----|----------------|

覚え書き

タスクコンテキスト内から呼び出す必要があります。

**6.1.3.10 start()**

```
void tk::start ()
```

tinykernel スケジューラを起動します

Core 0 のスケジューラを開始し、Core 1 を自動ブートしてデュアルコア並行実行を開始します。この関数から制御が戻ることはありません (Never returns)。

**6.1.3.11 unlock()**

```
void tk::unlock (  
    bool was_enabled)
```

tk::lock() (p.14) で入ったクリティカルセクションを抜けます

## 引数

|             |                         |
|-------------|-------------------------|
| was_enabled | tk::lock() (p. 14) の戻り値 |
|-------------|-------------------------|

### 6.1.3.12 yield()

```
void tk::yield ()
```

現在実行中のタスクを一時中断し、同優先度の他のタスクへ CPU を譲渡します

覚え書き

タスクコンテキスト内から呼び出す必要があります。

## 6.2 tk::detail 名前空間

### 関数

- void static\_register (const char \*name, TaskFn fn, uint32\_t prio)
- uint32\_t sched\_lock ()
- void sched\_unlock (uint32\_t save)
- void block\_current\_task\_locked ()
- void wake\_task\_locked ( TaskHandle handle)
- uint8\_t current\_task\_slot ()
- uint8\_t find\_highest\_prio\_waiter ( task\_mask\_t wait\_mask)

### 6.2.1 関数詳解

#### 6.2.1.1 block\_current\_task\_locked()

```
void tk::detail::block_current_task_locked ()
```

#### 6.2.1.2 current\_task\_slot()

```
uint8_t tk::detail::current_task_slot ()
```

#### 6.2.1.3 find\_highest\_prio\_waiter()

```
uint8_t tk::detail::find_highest_prio_waiter (  
    task_mask_t wait_mask)
```

#### 6.2.1.4 sched\_lock()

```
uint32_t tk::detail::sched_lock ()
```

#### 6.2.1.5 sched\_unlock()

```
void tk::detail::sched_unlock (  
    uint32_t save)
```

#### 6.2.1.6 static\_register()

```
void tk::detail::static_register (  
    const char * name,  
    TaskFn fn,  
    uint32_t prio)
```

#### 6.2.1.7 wake\_task\_locked()

```
void tk::detail::wake_task_locked (  
    TaskHandle handle)
```

## Chapter 7

# クラス詳解

### 7.1 tk::Actor< MsgType, QueueSize > クラステンプレート

メッセージ駆動型タスク (アクター) の基底クラス

```
#include <actor.hpp>
```

#### 公開メンバ関数

- Actor ()=default
- virtual ~Actor ()
- bool start (uint32\_t prio= PRIO\_NORMAL)  
Actor (p. 19) をタスクとして生成し、EventHub へ自動登録します
- bool post (const MsgType &msg)  
この Actor (p. 19) のメッセージキューへ直接メッセージを送信します (コピー渡し)
- bool post (MsgType &&msg)  
この Actor (p. 19) のメッセージキューへ直接メッセージを送信します (ムーブ渡し)
- template<typename U = MsgType>  
std::enable\_if< std::is\_same< U, Message >::value, bool >::type post (uint32\_t id, uintptr\_t arg=0)  
tk::Message (p. 26) 型の場合のコンビニエンス送信関数
- TaskHandle handle () const

#### 限定公開メンバ関数

- virtual void on\_start ()  
タスク起動時、イベントループ突入直前に 1 度だけ呼ばれる初期化関数
- virtual void on\_message (const MsgType &msg)=0  
メッセージ受信時に呼ばれるハンドラ関数

#### 7.1.1 詳解

---

```
template<typename MsgType = Message, size_t QueueSize = 8>  
class tk::Actor< MsgType, QueueSize >
```

メッセージ駆動型タスク (アクター) の基底クラス

## テンプレート引数

|           |                                          |
|-----------|------------------------------------------|
| MsgType   | 受信するメッセージの型 (デフォルト: tk::Message (p. 26)) |
| QueueSize | メッセージキューの最大保持件数 (デフォルト: 8)               |

```
struct SensorMsg { float temp; };

class SensorActor : public tk::Actor<SensorMsg, 8> {
protected:
    void on_start() override {
        // デバイスの初期化
    }
    void on_message(const SensorMsg& msg) override {
        printf("Temp: %.1f\n", msg.temp);
    }
};
```

### 7.1.2 構築子と解体子

#### 7.1.2.1 Actor()

```
template<typename MsgType = Message, size_t QueueSize = 8>
tk::Actor< MsgType, QueueSize >::Actor () [default]
```

#### 7.1.2.2 ~Actor()

```
template<typename MsgType = Message, size_t QueueSize = 8>
virtual tk::Actor< MsgType, QueueSize >::~~ Actor () [inline], [virtual]
```

### 7.1.3 関数詳解

#### 7.1.3.1 handle()

```
template<typename MsgType = Message, size_t QueueSize = 8>
TaskHandle tk::Actor< MsgType, QueueSize >::handle () const [inline]
```

この Actor (p. 19) のタスクハンドルを取得

#### 7.1.3.2 on\_message()

```
template<typename MsgType = Message, size_t QueueSize = 8>
virtual void tk::Actor< MsgType, QueueSize >::on_message (
    const MsgType & msg) [protected], [pure virtual]
```

メッセージ受信時に呼ばれるハンドラ関数

キューから取り出されたメッセージの実体への参照が渡されます。派生クラスで必ず実装します。

**引数**

|     |           |
|-----|-----------|
| msg | 受信したメッセージ |
|-----|-----------|

## 7.1.3.3 on\_start()

```
template<typename MsgType = Message, size_t QueueSize = 8>
virtual void tk::Actor< MsgType, QueueSize >::on_start () [inline], [protected], [virtual]
```

タスク起動時、イベントループ突入直前に 1 度だけ呼ばれる初期化関数

タスクコンテキスト内で実行されるため、tk::sleep\_ms() (p. 16) を使った初期化ウェイトも安全に行えます。必要に応じて派生クラスでオーバーライドします。

## 7.1.3.4 post() [1/3]

```
template<typename MsgType = Message, size_t QueueSize = 8>
bool tk::Actor< MsgType, QueueSize >::post (
    const MsgType & msg) [inline]
```

この Actor (p. 19) のメッセージキューへ直接メッセージを送信します (コピー渡し)

スレッドセーフ・割り込みセーフ・マルチコアセーフです。

**引数**

|     |           |
|-----|-----------|
| msg | 送信するメッセージ |
|-----|-----------|

戻り値

true 送信成功, false キューが満杯

## 7.1.3.5 post() [2/3]

```
template<typename MsgType = Message, size_t QueueSize = 8>
bool tk::Actor< MsgType, QueueSize >::post (
    MsgType && msg) [inline]
```

この Actor (p. 19) のメッセージキューへ直接メッセージを送信します (ムーブ渡し)

**引数**

|     |                 |
|-----|-----------------|
| msg | 送信するメッセージ (ムーブ) |
|-----|-----------------|

戻り値

true 送信成功, false キューが満杯

## 7.1.3.6 post() [3/3]

```
template<typename MsgType = Message, size_t QueueSize = 8>
template<typename U = MsgType>
std::enable_if< std::is_same< U, Message >::value, bool >::type tk::Actor< MsgType, QueueSize >::post (
    uint32_t id,
    uintptr_t arg = 0) [inline]
```

tk::Message (p. 26) 型の場合のコンビニエンス送信関数

## 引数

|     |             |
|-----|-------------|
| id  | メッセージID     |
| arg | 引数 (省略時: 0) |

## 戻り値

true 送信成功, false キューが満杯

## 7.1.3.7 start()

```
template<typename MsgType = Message, size_t QueueSize = 8>
bool tk::Actor< MsgType, QueueSize >::start (
    uint32_t prio = PRIO_NORMAL) [inline]
```

Actor (p. 19) をタスクとして生成し、EventHub へ自動登録します

tk::start() (p. 16) を呼び出す前に登録する必要があります。

## 引数

|      |                                       |
|------|---------------------------------------|
| prio | タスク優先度 (省略時: tk::PRIO_NORMAL (p. 13)) |
|------|---------------------------------------|

## 戻り値

true 登録成功, false 登録失敗 (タスク枠上限など)

このクラス詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ actor.hpp

## 7.2 tk::EventHub&lt; MsgType, MaxSubscribers &gt; クラステンプレート

メッセージ型ごとの超軽量 Pub/Sub デイスパッチャ

```
#include <actor.hpp>
```

## 静的公開メンバ関数

- static bool subscribe (void \*actor, bool(\*post\_fn)(void \*, const MsgType &))  
Actor (p. 19) をこのメッセージ型の購読者として登録します
- static bool unsubscribe (void \*actor)  
Actor (p. 19) の購読登録を解除します
- static size\_t publish (const MsgType &msg)  
このメッセージ型を購読しているすべての Actor (p. 19) へ同報配信します
- static size\_t subscriber\_count ()

### 7.2.1 詳解

```
template<typename MsgType, size_t MaxSubscribers = 4>
class tk::EventHub< MsgType, MaxSubscribers >
```

メッセージ型ごとの超軽量 Pub/Sub ディスパッチャ

テンプレート引数 `MsgType` ごとにコンパイル時解決される静的レジストリです。実行時の型検索オーバーヘッドゼロ、動的メモリ確保ゼロで動作します。

#### テンプレート引数

|                |                                             |
|----------------|---------------------------------------------|
| MsgType        | 配信するメッセージの型                                 |
| MaxSubscribers | 同一メッセージ型を購読できる最大 Actor (p. 19) 数 (デフォルト: 4) |

### 7.2.2 関数詳解

#### 7.2.2.1 publish()

```
template<typename MsgType, size_t MaxSubscribers = 4>
size_t tk::EventHub< MsgType, MaxSubscribers >::publish (
    const MsgType & msg) [inline], [static]
```

このメッセージ型を購読しているすべての Actor (p. 19) へ同報配信します

#### 引数

|     |                                         |
|-----|-----------------------------------------|
| msg | 配信するメッセージ実体 (各 Actor (p. 19) のキューへ値コピー) |
|-----|-----------------------------------------|

戻り値

size\_t 配信に成功した Actor (p. 19) の数

#### 7.2.2.2 subscribe()

構築: Doxygen

```
template<typename MsgType, size_t MaxSubscribers = 4>
bool tk::EventHub< MsgType, MaxSubscribers >::subscribe (
    void * actor,
    bool(* post_fn )(void *, const MsgType &)) [inline], [static]
```

**引数**

|         |                          |
|---------|--------------------------|
| actor   | Actor (p. 19) インスタンスポインタ |
| post_fn | メッセージ投入関数ポインタ            |

戻り値

true 登録成功, false 購読者上限に達した

**7.2.2.3 subscriber\_count()**

```
template<typename MsgType, size_t MaxSubscribers = 4>
size_t tk::EventHub< MsgType, MaxSubscribers >::subscriber_count () [inline], [static]
```

現在の購読者数を取得

**7.2.2.4 unsubscribe()**

```
template<typename MsgType, size_t MaxSubscribers = 4>
bool tk::EventHub< MsgType, MaxSubscribers >::unsubscribe (
    void * actor) [inline], [static]
```

Actor (p. 19) の購読登録を解除します

**引数**

|       |                               |
|-------|-------------------------------|
| actor | 解除する Actor (p. 19) インスタンスポインタ |
|-------|-------------------------------|

戻り値

true 解除成功, false 未登録

このクラス詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ actor.hpp

**7.3 FpuCtx 構造体****公開変数類**

- float s [32]

### 7.3.1 メンバ詳解

#### 7.3.1.1 s

```
float FpuCtx::s[32]
```

この構造体詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/ scheduler.cpp

## 7.4 tk::LockGuard< Lockable > クラステンプレート

ミューテックスのロックをスコープ単位で安全に管理する RAII クラス (std::lock\_guard 相当)

```
#include <sync.hpp>
```

### 公開メンバ関数

- LockGuard (Lockable &m)  
ロック対象を受け取り、即座に lock() (p. 14) します
- ~LockGuard ()
- LockGuard (const LockGuard &)=delete
- LockGuard & operator= (const LockGuard &)=delete

### 7.4.1 詳解

```
template<typename Lockable>
class tk::LockGuard< Lockable >
```

ミューテックスのロックをスコープ単位で安全に管理する RAII クラス (std::lock\_guard 相当)

構築時に lock() (p. 14) を呼び、デストラクタで自動的に unlock() (p. 16) を呼び出します。途中で return や例外が発生しても確実にロックが解放されます。

### テンプレート引数

|          |                                                               |
|----------|---------------------------------------------------------------|
| Lockable | lock() (p. 14) と unlock() (p. 16) を持つ型 (例: tk::Mutex (p. 27)) |
|----------|---------------------------------------------------------------|

### 7.4.2 構築子と解体子

#### 7.4.2.1 LockGuard() [1/2]

```
template<typename Lockable>
tk::LockGuard< Lockable >::LockGuard (
    Lockable & m) [inline], [explicit]
```

ロック対象を受け取り、即座に lock() (p. 14) します

**引数**

|   |                 |
|---|-----------------|
| m | ロック対象のミューテックス参照 |
|---|-----------------|

## 7.4.2.2 ~LockGuard()

```
template<typename Lockable>
tk::LockGuard< Lockable >::~LockGuard () [inline]
```

スコープ終了時に自動的に unlock() (p. 16) します

## 7.4.2.3 LockGuard() [2/2]

```
template<typename Lockable>
tk::LockGuard< Lockable >::~LockGuard (
    const LockGuard< Lockable > & ) [delete]
```

**7.4.3 関数詳解**

## 7.4.3.1 operator=()

```
template<typename Lockable>
LockGuard & tk::LockGuard< Lockable >::operator= (
    const LockGuard< Lockable > & ) [delete]
```

このクラス詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ sync.hpp

**7.5 tk::Message 構造体**

デフォルトの軽量メッセージ構造体

```
#include <actor.hpp>
```

**公開変数類**

- uint32\_t id  
メッセージ識別子 / コマンドID
- uintptr\_t arg  
引数 (ポインタまたは整数値)

### 7.5.1 詳解

デフォルトの軽量メッセージ構造体

### 7.5.2 メンバ詳解

#### 7.5.2.1 arg

```
uintptr_t tk::Message::arg
```

引数 (ポインタまたは整数値)

#### 7.5.2.2 id

```
uint32_t tk::Message::id
```

メッセージ識別子 / コマンドID

この構造体詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ actor.hpp

## 7.6 tk::Mutex クラス

所有権・優先度順ウェイクアップ・RAII 対応のミューテックス (排他ロック)

```
#include <sync.hpp>
```

### 公開メンバ関数

- Mutex ()=default
- void lock ()  
ミューテックスをロックします (空くまでブロック待機)
- bool try\_lock ()  
ミューテックスのロックを試みます (ノンブロッキング)
- void unlock ()  
ミューテックスのロックを解除します
- bool is\_locked () const

### 7.6.1 詳解

所有権・優先度順ウェイクアップ・RAII 対応のミューテックス (排他ロック)

共有リソースやペリフェラルへの同時アクセスを防止します。ロックしたタスク本人以外の誤った `unlock()` (p. 29) は安全に無視されます。C++ の `BasicLockable` を満たしており、`std::lock_guard` や `tk::LockGuard` (p. 25) が利用可能です。

```
tk::Mutex mtx;

{
    tk::LockGuard<tk::Mutex> lock(mtx);
    // クリティカルセクション...
} // 自動でアンロック
```

### 7.6.2 構築子と解体子

#### 7.6.2.1 Mutex()

```
tk::Mutex::Mutex () [default]
```

### 7.6.3 関数詳解

#### 7.6.3.1 is\_locked()

```
bool tk::Mutex::is_locked () const [inline]
```

現在ロックされているかどうかを判定

#### 7.6.3.2 lock()

```
void tk::Mutex::lock () [inline]
```

ミューテックスをロックします (空くまでブロック待機)

他のタスクが保持している場合は、解放されるまで自タスクを `State::Blocked` にして休止します。

#### 7.6.3.3 try\_lock()

```
bool tk::Mutex::try_lock () [inline]
```

ミューテックスのロックを試みます (ノンブロッキング)

戻り値

`true` ロック獲得成功

`false` 他のタスクが保持しているため獲得失敗 (ブロックせず即座に復帰)

## 7.6.3.4 unlock()

```
void tk::Mutex::unlock () [inline]
```

ミューテックスのロックを解除します

ロックを保持しているタスク本人だけが解除できます。待機中のタスクが存在する場合、最も優先度が高いタスクを 1 つ起床させます。

このクラス詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ sync.hpp

## 7.7 tk::PreemptGuard クラス

プリエンブション禁止をスコープ単位で安全に管理する RAII クラス

```
#include <kernel.hpp>
```

## 公開メンバ関数

- PreemptGuard ()
- ~PreemptGuard ()
- PreemptGuard (const PreemptGuard &)=delete
- PreemptGuard & operator= (const PreemptGuard &)=delete

## 7.7.1 詳解

プリエンブション禁止をスコープ単位で安全に管理する RAII クラス

生成時に自動で tk::preempt\_disable() (p. 14) を呼び、スコープを抜けた際に自動で tk::preempt\_↵enable() (p. 15) を呼びます。

```
{
    tk::PreemptGuard guard;
    // この区間はタスクスイッチが発生しない (I2C/SPI 通信などに最適)
    i2c_write_blocking(...);
} // 自動でタスクスイッチが再開
```

## 7.7.2 構築子と解体子

## 7.7.2.1 PreemptGuard() [1/2]

```
tk::PreemptGuard::PreemptGuard () [inline]
```

## 7.7.2.2 ~PreemptGuard()

```
tk::PreemptGuard::~~PreemptGuard () [inline]
```

### 7.7.2.3 PreemptGuard() [2/2]

```
tk::PreemptGuard::PreemptGuard (
    const PreemptGuard & ) [delete]
```

## 7.7.3 関数詳解

### 7.7.3.1 operator=()

```
PreemptGuard & tk::PreemptGuard::operator= (
    const PreemptGuard & ) [delete]
```

このクラス詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ kernel.hpp

## 7.8 tk::Semaphore クラス

計数セマフォ (Counting Semaphore (p. 30)) / バイナリセマフォ

```
#include <sync.hpp>
```

### 公開メンバ関数

- Semaphore (int32\_t initial\_count=1, int32\_t max\_count=1)  
セマフォを初期化します
- void acquire ()  
資源を獲得するまで現在のタスクをブロックします
- bool try\_acquire ()  
資源の獲得を試みます (ノンブロッキング)
- void release ()  
資源を返却し、待機中タスクがあれば最優先度のタスクを 1 つ起床させます
- int32\_t count () const

### 7.8.1 詳解

計数セマフォ (Counting Semaphore (p. 30)) / バイナリセマフォ

資源の獲得・返却によるタスク間同期を行います。資源がない場合はタスクが State::Blocked に遷移して休止し、返却時に優先度の高いタスクから順に起床します。

```
tk::Semaphore sem(0, 5); // 初期値 0、最大 5
```

```
// タスク側 (資源が空くまでブロック待機)
sem.acquire();
```

```
// シグナル送信側 (割り込みや他タスクから返却・通知)
sem.release();
```

### 7.8.2 構築子と解体子

## 引数

|               |                   |
|---------------|-------------------|
| initial_count | 初期資源カウント (省略時: 1) |
| max_count     | 最大資源カウント (省略時: 1) |

### 7.8.3 関数詳解

#### 7.8.3.1 acquire()

```
void tk::Semaphore::acquire () [inline]
```

資源を獲得するまで現在のタスクをブロックします

資源カウントが 0 以下の場合はタスクが休止状態になり、CPU は他タスクへ譲渡されます (CPU 消費ゼロ)。

#### 7.8.3.2 count()

```
int32_t tk::Semaphore::count () const [inline]
```

現在の資源カウントを取得

#### 7.8.3.3 release()

```
void tk::Semaphore::release () [inline]
```

資源を返却し、待機中タスクがあれば最優先度のタスクを 1 つ起床させます

#### 7.8.3.4 try\_acquire()

```
bool tk::Semaphore::try_acquire () [inline]
```

資源の獲得を試みます (ノンブロッキング)

戻り値

- true 獲得成功 (カウントが 1 減算された)
- false 資源がないため獲得失敗 (ブロックせず即座に復帰)

このクラス詳解は次のファイルから抽出されました:

- C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/ sync.hpp

## 7.9 tk::TaskHandle 構造体

タスクの識別ハンドル

```
#include <kernel.hpp>
```

### 公開メンバ関数

- `bool is_valid () const`

### 公開変数類

- `uint8_t slot {0xFF}`  
タスクスロット番号 (0xFF は無効/失敗)

### 7.9.1 詳解

タスクの識別ハンドル

### 7.9.2 関数詳解

#### 7.9.2.1 is\_valid()

```
bool tk::TaskHandle::is_valid () const [inline]
```

ハンドルが有効かどうかを判定

### 7.9.3 メンバ詳解

#### 7.9.3.1 slot

```
uint8_t tk::TaskHandle::slot {0xFF}
```

タスクスロット番号 (0xFF は無効/失敗)

この構造体詳解は次のファイルから抽出されました:

- `C:/Users/yoneda/Documents/Pico_Projects/tinykernel/tinykernel/include/tinykernel/ kernel.hpp`

## Chapter 8

# ファイル詳解

### 8.1 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵ tinykernel/include/tinykernel/actor.hpp **ファイル**

```
#include "kernel.hpp"  
#include <cstdint>  
#include <cstddef>  
#include <type_traits>  
#include <utility>
```

#### クラス

- struct tk::Message  
デフォルトの軽量メッセージ構造体
- class tk::EventHub< MsgType, MaxSubscribers >  
メッセージ型ごとの超軽量 Pub/Sub ディスパッチャ
- class tk::Actor< MsgType, QueueSize >  
メッセージ駆動型タスク (アクター) の基底クラス

#### 名前空間

- namespace tk

#### 関数

- template<typename MsgType>  
size\_t tk::publish (const MsgType &msg)  
メッセージの型を指定して、購読しているすべての Actor (p. 19) へ同報配信します
- size\_t tk::publish (uint32\_t id, uintptr\_t arg=0)  
tk::Message (p. 26) 型の同報配信ショートカット

## 8.2 actor.hpp

### [詳解]

```

00001 #pragma once
00002
00003 #include "kernel.hpp"
00004 #include <cstdint>
00005 #include <cstddef>
00006 #include <type_traits>
00007 #include <utility>
00008
00009 namespace tk {
00010
00011 struct Message {
00012     uint32_t id;
00013     uintptr_t arg;
00014 };
00015
00016 template <typename MsgType, size_t MaxSubscribers = 4>
00017 class EventHub {
00018     struct Subscriber {
00019         void* actor{nullptr};
00020         bool (*post_fn)(void*, const MsgType&){nullptr};
00021     };
00022
00023     static inline Subscriber subs_[MaxSubscribers]{};
00024     static inline size_t count_{0};
00025
00026 public:
00027     static bool subscribe(void* actor, bool (*post_fn)(void*, const MsgType&)) {
00028         uint32_t save = detail::sched_lock();
00029         if (count_ >= MaxSubscribers) {
00030             detail::sched_unlock(save);
00031             return false;
00032         }
00033         for (size_t i = 0; i < count_; ++i) {
00034             if (subs_[i].actor == actor) {
00035                 detail::sched_unlock(save);
00036                 return true; // 登録済み
00037             }
00038         }
00039         subs_[count_++] = {actor, post_fn};
00040         detail::sched_unlock(save);
00041         return true;
00042     }
00043
00044     static bool unsubscribe(void* actor) {
00045         uint32_t save = detail::sched_lock();
00046         for (size_t i = 0; i < count_; ++i) {
00047             if (subs_[i].actor == actor) {
00048                 for (size_t j = i; j + 1 < count_; ++j) {
00049                     subs_[j] = subs_[j + 1];
00050                 }
00051                 count_--;
00052                 detail::sched_unlock(save);
00053                 return true;
00054             }
00055         }
00056         detail::sched_unlock(save);
00057         return false;
00058     }
00059
00060     static size_t publish(const MsgType& msg) {
00061         Subscriber local_subs[MaxSubscribers];
00062         size_t n = 0;
00063         {
00064             uint32_t save = detail::sched_lock();
00065             n = count_;
00066             for (size_t i = 0; i < n; ++i) {
00067                 local_subs[i] = subs_[i];
00068             }
00069             detail::sched_unlock(save);
00070         }
00071
00072         size_t delivered = 0;
00073         for (size_t i = 0; i < n; ++i) {
00074             if (local_subs[i].post_fn && local_subs[i].post_fn(local_subs[i].actor, msg)) {
00075                 delivered++;
00076             }
00077         }
00078         return delivered;
00079     }
00080
00081     static size_t subscriber_count() {

```

```

00112     uint32_t save = detail::sched_lock();
00113     size_t n = count_;
00114     detail::sched_unlock(save);
00115     return n;
00116 }
00117 };
00118
00134 template <typename MsgType>
00135 inline size_t publish(const MsgType& msg) {
00136     return EventHub<MsgType>::publish(msg);
00137 }
00138
00146 inline size_t publish(uint32_t id, uintptr_t arg = 0) {
00147     return publish(Message{id, arg});
00148 }
00149
00173 template <typename MsgType = Message, size_t QueueSize = 8>
00174 class Actor {
00175     static_assert(QueueSize > 0, "QueueSize must be greater than 0");
00176
00177 public:
00178     Actor() = default;
00179     virtual ~Actor() {
00180         EventHub<MsgType>::unsubscribe(this);
00181     }
00182
00191     bool start(uint32_t prio = PRIO_NORMAL) {
00192         EventHub<MsgType>::subscribe(this, [](void* obj, const MsgType& m) {
00193             return static_cast<Actor<MsgType, QueueSize>*>(obj)->post(m);
00194         });
00195         handle_ = tk::create(task_entry, this, prio);
00196         return handle_.is_valid();
00197     }
00198
00207     bool post(const MsgType& msg) {
00208         uint32_t save = detail::sched_lock();
00209         if (count_ >= QueueSize) {
00210             detail::sched_unlock(save);
00211             return false;
00212         }
00213         queue_[tail_] = msg;
00214         tail_ = (tail_ + 1) % QueueSize;
00215         count_++;
00216         if (waiting_) {
00217             waiting_ = false;
00218             detail::wake_task_locked(handle_);
00219         }
00220         detail::sched_unlock(save);
00221         return true;
00222     }
00223
00230     bool post(MsgType&& msg) {
00231         uint32_t save = detail::sched_lock();
00232         if (count_ >= QueueSize) {
00233             detail::sched_unlock(save);
00234             return false;
00235         }
00236         queue_[tail_] = std::move(msg);
00237         tail_ = (tail_ + 1) % QueueSize;
00238         count_++;
00239         if (waiting_) {
00240             waiting_ = false;
00241             detail::wake_task_locked(handle_);
00242         }
00243         detail::sched_unlock(save);
00244         return true;
00245     }
00246
00254     template <typename U = MsgType>
00255     typename std::enable_if<std::is_same<U, Message>::value, bool>::type
00256     post(uint32_t id, uintptr_t arg = 0) {
00257         return post(Message{id, arg});
00258     }
00259
00261     TaskHandle handle() const { return handle_; }
00262
00263 protected:
00270     virtual void on_start() {}
00271
00280     virtual void on_message(const MsgType& msg) = 0;
00281
00282 private:
00283     static void task_entry(void* arg) {
00284         static_cast<Actor<MsgType, QueueSize>*>(arg)->event_loop();
00285     }
00286
00287     void event_loop() {

```

```

00288     on_start();
00289     for (;;) {
00290         MsgType msg{};
00291         uint32_t save = detail::sched_lock();
00292         while (count_ == 0) {
00293             waiting_ = true;
00294             detail::block_current_task_locked();
00295             detail::sched_unlock(save);
00296             // コンテキストスイッチされ、他のタスクが走る。
00297             // wake_task_locked で起こされたらここに戻る。
00298             save = detail::sched_lock();
00299         }
00300         msg = std::move(queue_[head_]);
00301         head_ = (head_ + 1) % QueueSize;
00302         count_--;
00303         waiting_ = false;
00304         detail::sched_unlock(save);
00305
00306         // メッセージディスパッチ（ロック解除状態で実行）
00307         on_message(msg);
00308     }
00309 }
00310
00311 MsgType queue_[QueueSize]{};
00312 size_t head_{0};
00313 size_t tail_{0};
00314 size_t count_{0};
00315 bool waiting_{false};
00316 TaskHandle handle_{};
00317 };
00318
00319 } // namespace tk

```

### 8.3 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/← tinykernel/include/tinykernel/kernel.hpp **ファイル**

```

#include <stdint.h>
#include <stddef.h>
#include "pico/runtime.h"

```

#### クラス

- struct tk::TaskHandle  
タスクの識別ハンドル
- class tk::PreemptGuard  
プリエンプション禁止をスコープ単位で安全に管理する RAII クラス

#### 名前空間

- namespace tk
- namespace tk::detail

#### マクロ定義

- #define TK\_TICKS\_PER\_SEC 1000u
- #define TK\_TIME\_SLICE\_MS 10u
- #define TK\_MAX\_TASKS 32u
- #define TK\_DEFAULT\_STACK\_SIZE (4 \* 1024)
- #define TK\_TASK(fn, prio)  
リンク時にタスクを自動登録するマクロ（静的タスク登録）

## 型定義

- using tk::TaskFn = void (\*)(void\* arg)
- using tk::task\_mask\_t = uint32\_t

## 列挙型

- enum tk::Prio : uint32\_t { tk::PRIO\_REALTIME = 1 , tk::PRIO\_HIGH = 5 , tk::PRIO\_↵  
NORMAL = 9 , tk::PRIO\_LOW = 13 }  
タスクの優先度レベル

## 関数

- TaskHandle tk::create ( TaskFn fn, void \*arg=nullptr, uint32\_t prio= PRIO\_NORMAL)  
タスクを生成・登録します
- void tk::start ()  
tinykernel スケジューラを起動します
- void tk::yield ()  
現在実行中のタスクを一時中断し、同優先度の他のタスクへ CPU を譲渡します
- void tk::sleep\_ms (uint32\_t ms)  
現在のタスクを指定時間スリープさせます
- void tk::exit (int code=0)  
現在のタスクを終了・破棄します
- uint64\_t tk::now\_ms ()  
システム起動時からの経過時間をミリ秒単位で取得します
- bool tk::lock ()  
ハードウェアスピンロックと全割り込み禁止によるクリティカルセクションに入ります
- void tk::unlock (bool was\_enabled)  
tk::lock() (p.14) で入ったクリティカルセクションを抜けます
- void tk::preempt\_disable ()  
自コアのタスクスイッチ (プリエンプション) を一時的に禁止します
- void tk::preempt\_enable ()  
自コアのタスクスイッチ (プリエンプション) を再開します
- void tk::detail::static\_register (const char \*name, TaskFn fn, uint32\_t prio)
- uint32\_t tk::detail::sched\_lock ()
- void tk::detail::sched\_unlock (uint32\_t save)
- void tk::detail::block\_current\_task\_locked ()
- void tk::detail::wake\_task\_locked ( TaskHandle handle)
- uint8\_t tk::detail::current\_task\_slot ()
- uint8\_t tk::detail::find\_highest\_prio\_waiter ( task\_mask\_t wait\_mask)

### 8.3.1 マクロ定義詳解

#### 8.3.1.1 TK\_DEFAULT\_STACK\_SIZE

```
#define TK_DEFAULT_STACK_SIZE (4 * 1024)
```

1 タスクあたりのスタックサイズ (バイト)

### 8.3.1.2 TK\_MAX\_TASKS

```
#define TK_MAX_TASKS 32u
```

最大タスク数 (スロット 0 はCore0 アイドル、スロット 1 はCore1 アイドル専用)

### 8.3.1.3 TK\_TASK

```
#define TK_TASK(
    fn,
    prio)
```

**値:**

```
static void tk_register_##fn(void) {
    ::tk::detail::static_register(#fn, fn, (prio));
}
PICO_RUNTIME_INIT_FUNC(tk_register_##fn, "ZZ900");
```

リンク時にタスクを自動登録するマクロ (静的タスク登録)

main() の開始前に Pico SDK のランタイムイニシャライザを利用してタスクを登録します。

#### 引数

|      |                                     |
|------|-------------------------------------|
| fn   | タスク関数名 (void fn(void* arg))         |
| prio | タスク優先度 (例: tk::PRIO_NORMAL (p. 13)) |

### 8.3.1.4 TK\_TICKS\_PER\_SEC

```
#define TK_TICKS_PER_SEC 1000u
```

1 秒あたりのタイマースケジューラ tick 数 (Hz)

### 8.3.1.5 TK\_TIME\_SLICE\_MS

```
#define TK_TIME_SLICE_MS 10u
```

同一優先度タスク間のタイムスライス時間 (ミリ秒)

## 8.4 kernel.hpp

**[詳解]**

```

00001 // tinykernel: minimal preemptive task scheduler for RP2350 (Pico 2, Cortex-M33)
00002 #pragma once
00003
00004 #include <stdint.h>
00005 #include <stddef.h>
00006
00007 #include "pico/runtime.h" // PICO_RUNTIME_INIT_FUNC (static task registration)
00008
00010 #ifndef TK_TICKS_PER_SEC
00011 #define TK_TICKS_PER_SEC 1000u
00012 #endif
00013
00015 #ifndef TK_TIME_SLICE_MS
00016 #define TK_TIME_SLICE_MS 10u
00017 #endif
00018
00020 #ifndef TK_MAX_TASKS
00021 #define TK_MAX_TASKS 32u
00022 #endif
00023
00025 #ifndef TK_DEFAULT_STACK_SIZE
00026 #define TK_DEFAULT_STACK_SIZE (4 * 1024)
00027 #endif
00028
00029 namespace tk {
00030
00032 using TaskFn = void (*)(void* arg);
00033
00039 enum Prio : uint32_t {
00040     PRIO_REALTIME = 1,
00041     PRIO_HIGH     = 5,
00042     PRIO_NORMAL   = 9,
00043     PRIO_LOW      = 13,
00044 };
00045
00049 struct TaskHandle {
00050     uint8_t slot{0xFF};
00051
00053     bool is_valid() const { return slot != 0xFF; }
00054 };
00055
00066 TaskHandle create(TaskFn fn, void* arg = nullptr, uint32_t prio = PRIO_NORMAL);
00067
00074 void start();
00075
00081 void yield();
00082
00091 void sleep_ms(uint32_t ms);
00092
00099 void exit(int code = 0);
00100
00106 uint64_t now_ms();
00107
00115 bool lock();
00116
00122 void unlock(bool was_enabled);
00123
00132 void preempt_disable();
00133
00141 void preempt_enable();
00142
00156 class PreemptGuard {
00157 public:
00158     PreemptGuard() { preempt_disable(); }
00159     ~PreemptGuard() { preempt_enable(); }
00160
00161     PreemptGuard(const PreemptGuard&) = delete;
00162     PreemptGuard& operator=(const PreemptGuard&) = delete;
00163 };
00164
00165 using task_mask_t = uint32_t;
00166
00167 namespace detail {
00168 void static_register(const char* name, TaskFn fn, uint32_t prio);
00169 uint32_t sched_lock();
00170 void sched_unlock(uint32_t save);
00171 void block_current_task_locked();
00172 void wake_task_locked(TaskHandle handle);
00173 uint8_t current_task_slot();
00174 uint8_t find_highest_prio_waiter(task_mask_t wait_mask);
00175 } // namespace detail

```

```

00176
00177 } // namespace tk
00178
00187 #define TK_TASK(fn, prio) \
00188     static void tk_register_##fn(void) { \
00189         ::tk::detail::static_register(#fn, fn, (prio)); \
00190     } \
00191     PICO_RUNTIME_INIT_FUNC(tk_register_##fn, "ZZ900");

```

## 8.5 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/⇐ tinykernel/include/tinykernel/sync.hpp **ファイル**

```

#include "kernel.hpp"
#include <cstdint>

```

### クラス

- class tk::Semaphore  
計数セマフォ (Counting Semaphore (p. 30)) / バイナリセマフォ
- class tk::Mutex  
所有権・優先度順ウェイクアップ・RAII 対応のミューテックス (排他ロック)
- class tk::LockGuard< Lockable >  
ミューテックスのロックをスコープ単位で安全に管理する RAII クラス (std::lock\_guard 相当)

### 名前空間

- namespace tk

### 型定義

- using tk::BinarySemaphore = Semaphore  
バイナリセマフォ (初期値 1, 最大 1) の型エイリアス

## 8.6 sync.hpp

### [詳解]

```

00001 #pragma once
00002
00003 #include "kernel.hpp"
00004 #include <cstdint>
00005
00006 namespace tk {
00007
00024 class Semaphore {
00025 public:
00031     explicit Semaphore(int32_t initial_count = 1, int32_t max_count = 1)
00032         : count_(initial_count), max_count_(max_count) {}
00033
00039     void acquire() {
00040         uint32_t save = detail::sched_lock();
00041         while (count_ <= 0) {
00042             uint8_t slot = detail::current_task_slot();
00043             wait_mask_ |= (task_mask_t{1} << slot);
00044             detail::block_current_task_locked();

```

```

00045     detail::sched_unlock(save);
00046     // 他タスクから release() されて再開したら再チェック
00047     save = detail::sched_lock();
00048 }
00049 count--;
00050 detail::sched_unlock(save);
00051 }
00052
00053 bool try_acquire() {
00054     uint32_t save = detail::sched_lock();
00055     bool success = false;
00056     if (count_ > 0) {
00057         count--;
00058         success = true;
00059     }
00060     detail::sched_unlock(save);
00061     return success;
00062 }
00063
00064 void release() {
00065     uint32_t save = detail::sched_lock();
00066     if (count_ < max_count_) {
00067         count_++;
00068     }
00069     if (wait_mask_ != 0) {
00070         uint8_t slot = detail::find_highest_prio_waiter(wait_mask_);
00071         if (slot != 0xFF) {
00072             wait_mask_ &= ~(task_mask_t{1} « slot);
00073             detail::wake_task_locked(TaskHandle{slot});
00074         }
00075     }
00076     detail::sched_unlock(save);
00077 }
00078
00079 int32_t count() const {
00080     uint32_t save = detail::sched_lock();
00081     int32_t c = count_;
00082     detail::sched_unlock(save);
00083     return c;
00084 }
00085
00086 private:
00087     int32_t count_;
00088     int32_t max_count_;
00089     task_mask_t wait_mask_{0};
00090 };
00091
00092 using BinarySemaphore = Semaphore;
00093
00094 class Mutex {
00095 public:
00096     Mutex() = default;
00097
00098     void lock() {
00099         uint32_t save = detail::sched_lock();
00100         uint8_t my_slot = detail::current_task_slot();
00101         while (locked_) {
00102             wait_mask_ |= (task_mask_t{1} « my_slot);
00103             detail::block_current_task_locked();
00104             detail::sched_unlock(save);
00105             save = detail::sched_lock();
00106         }
00107         locked_ = true;
00108         owner_slot_ = my_slot;
00109         detail::sched_unlock(save);
00110     }
00111
00112     bool try_lock() {
00113         uint32_t save = detail::sched_lock();
00114         uint8_t my_slot = detail::current_task_slot();
00115         bool success = false;
00116         if (!locked_) {
00117             locked_ = true;
00118             owner_slot_ = my_slot;
00119             success = true;
00120         }
00121         detail::sched_unlock(save);
00122         return success;
00123     }
00124
00125     void unlock() {
00126         uint32_t save = detail::sched_lock();
00127         uint8_t my_slot = detail::current_task_slot();
00128         if (locked_ && owner_slot_ == my_slot) {
00129             locked_ = false;
00130             owner_slot_ = 0xFF;
00131             if (wait_mask_ != 0) {

```

```

00178         uint8_t slot = detail::find_highest_prio_waiter(wait_mask_);
00179         if (slot != 0xFF) {
00180             wait_mask_ &= ~(task_mask_t{1} « slot);
00181             detail::wake_task_locked(TaskHandle{slot});
00182         }
00183     }
00184 }
00185 detail::sched_unlock(save);
00186 }
00187
00188 bool is_locked() const {
00189     uint32_t save = detail::sched_lock();
00190     bool lk = locked_;
00191     detail::sched_unlock(save);
00192     return lk;
00193 }
00194 }
00195
00196 private:
00197     bool locked_{false};
00198     uint8_t owner_slot_{0xFF};
00199     task_mask_t wait_mask_{0};
00200 };
00201
00210 template <typename Lockable>
00211 class LockGuard {
00212 public:
00217     explicit LockGuard(Lockable& m) : m_(m) { m_.lock(); }
00218
00220     ~LockGuard() { m_.unlock(); }
00221
00222     LockGuard(const LockGuard&) = delete;
00223     LockGuard& operator=(const LockGuard&) = delete;
00224
00225 private:
00226     Lockable& m_;
00227 };
00228
00229 } // namespace tk

```

## 8.7 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵ tinykernel/src/port/m33/port\_m33.c ファイル

```

#include <stdint.h>
#include "hardware/clocks.h"
#include "hardware/structs/clocks.h"
#include "hardware/structs/m33.h"
#include "hardware/structs/scb.h"
#include "hardware/structs/systick.h"
#include "port_m33.h"

```

### マクロ定義

- #define FPCCR\_ASPEN (1u << 31)
- #define FPCCR\_LSPEN (1u << 30)

### 関数

- void tk\_port\_init (uint32\_t ticks\_per\_sec)

## 8.7.1 マクロ定義詳解

### 8.7.1.1 FPCCR\_ASPEN

```
#define FPCCR_ASPEN (1u << 31)
```

### 8.7.1.2 FPCCR\_LSPEN

```
#define FPCCR_LSPEN (1u << 30)
```

## 8.7.2 関数詳解

### 8.7.2.1 tk\_port\_init()

```
void tk_port_init (  
    uint32_t ticks_per_sec)
```

## 8.8 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵ tinykernel/src/port/m33/port\_m33.h **ファイル**

```
#include <stdint.h>  
#include "hardware/structs/scb.h"
```

## 関数

- void tk\_port\_init (uint32\_t ticks\_per\_sec)
- uintptr\_t tk\_port\_make\_frame (uint8\_t \*base, uint32\_t size\_bytes, void \*entry\_fn, void \*arg)
- void tk\_port\_start (uintptr\_t sp)

## 8.8.1 関数詳解

### 8.8.1.1 tk\_port\_init()

```
void tk_port_init (  
    uint32_t ticks_per_sec)
```

### 8.8.1.2 tk\_port\_make\_frame()

```
uintptr_t tk_port_make_frame (  
    uint8_t * base,  
    uint32_t size_bytes,  
    void * entry_fn,  
    void * arg)
```

### 8.8.1.3 tk\_port\_start()

```
void tk_port_start (
    uintptr_t sp)
```

## 8.9 port\_m33.h

### [詳解]

```
00001 #pragma once
00002
00003 #include <stdint.h>
00004
00005 #include "hardware/structs/scb.h"
00006
00007 #ifdef __cplusplus
00008 extern "C" {
00009 #endif
00010
00011 // One-time port setup: FPU context policy + SysTick + PendSV priority. Called from tk::start().
00012 void tk_port_init(uint32_t ticks_per_sec);
00013
00014 // Build a task's initial stack frame. Returns the SP to start with.
00015 uintptr_t tk_port_make_frame(uint8_t* base, uint32_t size_bytes, void* entry_fn, void* arg);
00016
00017 // Jump into the first task (r0 = SP from tk_port_make_frame). Never returns.
00018 void tk_port_start(uintptr_t sp);
00019
00020 // Trigger a PendSV exception to initiate context switching.
00021 static inline void tk_port_trigger_pendsv(void) {
00022     scb_hw->icsr = 1u « 28; // SCB_ICSR_PENDSVSET_Msk
00023     __asm__ volatile("isb" ::: "memory");
00024 }
00025
00026 #ifdef __cplusplus
00027 }
00028 #endif
```

## 8.10 C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/↵ tinykernel/src/scheduler.cpp **ファイル**

```
#include "tinykernel/kernel.hpp"
#include <cstring>
#include "pico/platform.h"
#include "pico/multicore.h"
#include "hardware/sync.h"
#include "port/m33/port_m33.h"
```

### クラス

- struct FpuCtx

### 名前空間

- namespace tk
- namespace tk::detail

## 関数

- void isr\_systick (void)
- uintptr\_t tk\_switch\_context (uintptr\_t cur\_sp)
- void tk::detail::static\_register (const char \*name, TaskFn fn, uint32\_t prio)
- void tk\_task\_return\_trampoline (void)

## 変数

- FpuCtx tk\_fpu\_scratch [TK\_NUM\_CORES] = {}

### 8.10.1 関数詳解

#### 8.10.1.1 isr\_systick()

```
void isr_systick (  
    void )
```

#### 8.10.1.2 tk\_switch\_context()

```
uintptr_t tk_switch_context (  
    uintptr_t cur_sp)
```

#### 8.10.1.3 tk\_task\_return\_trampoline()

```
void tk_task_return_trampoline (  
    void )
```

### 8.10.2 変数詳解

#### 8.10.2.1 tk\_fpu\_scratch

```
FpuCtx tk_fpu_scratch[TK_NUM_CORES] = {}
```



# Index

~Actor  
    tk::Actor< MsgType, QueueSize >, 20  
~LockGuard  
    tk::LockGuard< Lockable >, 26  
~PreemptGuard  
    tk::PreemptGuard, 29  
acquire  
    tk::Semaphore, 31  
Actor  
    tk::Actor< MsgType, QueueSize >, 20  
arg  
    tk::Message, 27  
BinarySemaphore  
    tk, 12  
block\_current\_task\_locked  
    tk::detail, 17  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/  
    ディレクトリリファレンス, 10  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/  
    ディレクトリリファレンス, 9  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/  
    ディレクトリリファレンス, 10  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/actor.hpp,  
    33, 34  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/kernel.hpp,  
    36, 39  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/include/tinykernel/sync.hpp,  
    40  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/  
    ディレクトリリファレンス, 9  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/port/  
    ディレクトリリファレンス, 9  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/port/m33  
    ディレクトリリファレンス, 9  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/port/m33/port\_m33.c,  
    42  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/port/m33/port\_m33.h,  
    43, 44  
C:/Users/yoneda/Documents/Pico\_Projects/tinykernel/tinykernel/src/scheduler.cpp,  
    44  
count  
    tk::Semaphore, 31  
create  
    tk, 13  
current\_task\_slot  
    tk::detail, 17  
exit  
    tk, 13  
find\_highest\_prio\_waiter  
    tk::detail, 17  
FPCCR\_ASPEN  
    port\_m33.c, 43  
FPCCR\_LSPEN  
    port\_m33.c, 43  
FpuCtx, 24  
    s, 25  
handle  
    tk::Actor< MsgType, QueueSize >, 20  
id  
    tk::Message, 27  
is\_locked  
    tk::Mutex, 28  
is\_valid  
    tk::TaskHandle, 32  
isr\_systick  
    scheduler.cpp, 45  
kernel.hpp  
    TK\_DEFAULT\_STACK\_SIZE, 37  
    TK\_MAX\_TASKS, 37  
    TK\_TASK, 38  
    TK\_TICKS\_PER\_SEC, 38  
    TK\_TIME\_SLICE\_MS, 38  
lock  
    tk::detail, 17  
    tk::Mutex, 28  
    tk::LockGuard  
    tk::LockGuard< Lockable >, 25, 26  
    Mutex  
    tk::Mutex, 28  
now\_ms  
    tk, 14  
on\_message  
    tk::Actor< MsgType, QueueSize >, 20  
on\_start  
    tk::Actor< MsgType, QueueSize >, 21  
operator=  
    tk::LockGuard< Lockable >, 26  
    tk::PreemptGuard, 30  
port\_m33.c

- FPCCR\_ASPEN, 43
- FPCCR\_LSPEN, 43
- tk\_port\_init, 43
- port\_m33.h
  - tk\_port\_init, 43
  - tk\_port\_make\_frame, 43
  - tk\_port\_start, 43
- post
  - tk::Actor< MsgType, QueueSize >, 21
- preempt\_disable
  - tk, 14
- preempt\_enable
  - tk, 14
- PreemptGuard
  - tk::PreemptGuard, 29
- Prio
  - tk, 13
- PRIO\_HIGH
  - tk, 13
- PRIO\_LOW
  - tk, 13
- PRIO\_NORMAL
  - tk, 13
- PRIO\_REALTIME
  - tk, 13
- publish
  - tk, 15
  - tk::EventHub< MsgType, MaxSubscribers >, 23
- release
  - tk::Semaphore, 31
- s
  - FpuCtx, 25
- sched\_lock
  - tk::detail, 17
- sched\_unlock
  - tk::detail, 18
- scheduler.cpp
  - isr\_systick, 45
  - tk\_fpu\_scratch, 45
  - tk\_switch\_context, 45
  - tk\_task\_return\_trampoline, 45
- Semaphore
  - tk::Semaphore, 30
- sleep\_ms
  - tk, 16
- slot
  - tk::TaskHandle, 32
- start
  - tk, 16
  - tk::Actor< MsgType, QueueSize >, 22
- static\_register
  - tk::detail, 18
- subscribe
  - tk::EventHub< MsgType, MaxSubscribers >, 23
- subscriber\_count
- tk::EventHub< MsgType, MaxSubscribers >, 24
- task\_mask\_t
  - tk, 12
- TaskFn
  - tk, 12
- tk, 11
  - BinarySemaphore, 12
  - create, 13
  - exit, 13
  - lock, 14
  - now\_ms, 14
  - preempt\_disable, 14
  - preempt\_enable, 14
  - Prio, 13
  - PRIO\_HIGH, 13
  - PRIO\_LOW, 13
  - PRIO\_NORMAL, 13
  - PRIO\_REALTIME, 13
  - publish, 15
  - sleep\_ms, 16
  - start, 16
  - task\_mask\_t, 12
  - TaskFn, 12
  - unlock, 16
  - yield, 17
- tk::Actor< MsgType, QueueSize >, 19
  - ~Actor, 20
  - Actor, 20
  - handle, 20
  - on\_message, 20
  - on\_start, 21
  - post, 21
  - start, 22
- tk::detail, 17
  - block\_current\_task\_locked, 17
  - current\_task\_slot, 17
  - find\_highest\_prio\_waiter, 17
  - sched\_lock, 17
  - sched\_unlock, 18
  - static\_register, 18
  - wake\_task\_locked, 18
- tk::EventHub< MsgType, MaxSubscribers >, 22
  - publish, 23
  - subscribe, 23
  - subscriber\_count, 24
  - unsubscribe, 24
- tk::LockGuard< Lockable >, 25
  - ~LockGuard, 26
  - LockGuard, 25, 26
  - operator=, 26
- tk::Message, 26
  - arg, 27
  - id, 27
- tk::Mutex, 27
  - is\_locked, 28
  - lock, 28
  - Mutex, 28

- try\_lock, 28
- unlock, 28
- tk::PreemptGuard, 29
  - ~PreemptGuard, 29
  - operator=, 30
  - PreemptGuard, 29
- tk::Semaphore, 30
  - acquire, 31
  - count, 31
  - release, 31
  - Semaphore, 30
  - try\_acquire, 31
- tk::TaskHandle, 32
  - is\_valid, 32
  - slot, 32
- TK\_DEFAULT\_STACK\_SIZE
  - kernel.hpp, 37
- tk\_fpu\_scratch
  - scheduler.cpp, 45
- TK\_MAX\_TASKS
  - kernel.hpp, 37
- tk\_port\_init
  - port\_m33.c, 43
  - port\_m33.h, 43
- tk\_port\_make\_frame
  - port\_m33.h, 43
- tk\_port\_start
  - port\_m33.h, 43
- tk\_switch\_context
  - scheduler.cpp, 45
- TK\_TASK
  - kernel.hpp, 38
- tk\_task\_return\_trampoline
  - scheduler.cpp, 45
- TK\_TICKS\_PER\_SEC
  - kernel.hpp, 38
- TK\_TIME\_SLICE\_MS
  - kernel.hpp, 38
- try\_acquire
  - tk::Semaphore, 31
- try\_lock
  - tk::Mutex, 28
- unlock
  - tk, 16
  - tk::Mutex, 28
- unsubscribe
  - tk::EventHub< MsgType, MaxSubscribers >, 24
- wake\_task\_locked
  - tk::detail, 18
- yield
  - tk, 17